

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY  
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



## MACHINE LEARNING (CO3117)

---

Assignment Report CC01

# BLOOD CELL CLASSIFICATION

---

**Instructor:** Dr. Truong Vinh Lan

|                  |                     |         |
|------------------|---------------------|---------|
| <b>Students:</b> | Hoang Minh Cam Tu   | 2353289 |
|                  | Pham Hong Minh Tu   | 2353293 |
|                  | Thai Thien Hai Long | 2352699 |
|                  | Be Nguyen Dang Khoa | 2352548 |
|                  | Tran Nguyen Giap    | 2352284 |



## Table of contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction &amp; Objectives</b>                         | <b>2</b>  |
| 1.1      | Project Context . . . . .                                    | 2         |
| 1.2      | Dataset Overview . . . . .                                   | 2         |
| 1.3      | Primary Objectives and Scope . . . . .                       | 2         |
| <b>2</b> | <b>Exploratory Data Analysis (EDA)</b>                       | <b>4</b>  |
| 2.1      | Class Distribution . . . . .                                 | 4         |
| 2.2      | Data Integrity & Pixel Decoding . . . . .                    | 4         |
| 2.3      | Brightness & Contrast Analytics . . . . .                    | 4         |
| 2.4      | Morphology & RGB Profiling . . . . .                         | 5         |
| <b>3</b> | <b>Data Pre-Processing</b>                                   | <b>6</b>  |
| 3.1      | Stratified Splitting Protocol . . . . .                      | 6         |
| 3.2      | Geometric Transforms & Data Augmentation Pipelines . . . . . | 6         |
| 3.3      | Foundation Stage Conclusion . . . . .                        | 7         |
| <b>4</b> | <b>Traditional Machine Learning Pipeline</b>                 | <b>8</b>  |
| 4.1      | Feature Extraction . . . . .                                 | 8         |
| 4.1.1    | Histogram of Oriented Gradients (HOG) . . . . .              | 8         |
| 4.1.2    | SIFT with Bag-of-Visual-Words (BoVW) . . . . .               | 9         |
| 4.1.3    | Dimensionality Reduction with PCA . . . . .                  | 10        |
| <b>5</b> | <b>Deep Learning Features (Bonus Section)</b>                | <b>12</b> |
| 5.1      | Architectural Selection . . . . .                            | 12        |
| 5.2      | Network Modification and Implementation Details . . . . .    | 12        |
| 5.3      | Training Dynamics and Optimization Strategy . . . . .        | 12        |
| 5.4      | Empirical Results and Performance Analysis . . . . .         | 13        |
| 5.5      | Deep Feature Serialization and Hybrid Integration . . . . .  | 13        |
| <b>6</b> | <b>Experimental Results &amp; Comparative Analysis</b>       | <b>14</b> |
| 6.1      | Classifier Selection . . . . .                               | 14        |
| 6.1.1    | Support Vector Machine (SVM) . . . . .                       | 14        |
| 6.1.2    | Random Forest . . . . .                                      | 14        |
| 6.1.3    | k-Nearest Neighbors (k-NN) . . . . .                         | 15        |
| 6.1.4    | Logistic Regression . . . . .                                | 16        |
| 6.1.5    | Naive Bayes . . . . .                                        | 16        |
| 6.2      | Baseline Evaluation . . . . .                                | 17        |
| 6.3      | Hyperparameter Optimization . . . . .                        | 20        |
| 6.4      | Fine-Tuning . . . . .                                        | 21        |
| 6.5      | Comparative Analysis . . . . .                               | 23        |
| 6.5.1    | Multi-metric Comparative Bar Chart . . . . .                 | 23        |
| 6.5.2    | Accuracy Heatmap . . . . .                                   | 24        |
| 6.5.3    | Confusion Matrix Analysis . . . . .                          | 25        |
| <b>7</b> | <b>Conclusion &amp; Future Work</b>                          | <b>28</b> |
| 7.1      | Conclusion . . . . .                                         | 28        |
| 7.2      | Future Work . . . . .                                        | 28        |
| <b>8</b> | <b>Deliverables &amp; Artifacts</b>                          | <b>29</b> |
| 8.1      | GitHub Repository . . . . .                                  | 29        |
| 8.2      | Google Colab Notebooks . . . . .                             | 29        |
| 8.3      | Serialized Feature Arrays (.npy) . . . . .                   | 29        |
| 8.4      | Final Documentation . . . . .                                | 29        |

# 1 Introduction & Objectives

## 1.1 Project Context

In recent years, the intersection of computer vision and medical image analysis has catalyzed significant advancements in clinical diagnostics. The manual counting and morphological classification of white blood cells (leukocytes) via microscopic examination is a labor-intensive, time-consuming process highly susceptible to human error and visual fatigue. Consequently, developing robust, automated machine learning pipelines to process pathology image data is of paramount importance to modern biomedical engineering.

This project focuses on the automated classification of four core leukocyte sub-types (Eosinophils, Lymphocytes, Monocytes, and Neutrophils) which serve as the primary cellular defense mechanisms of the human immune system. Accurate classification forms the foundation for early detection of infections, immune disorders, and blood cancers.

## 1.2 Dataset Overview

In the scope of this course project, we utilize a specialized dataset of microscopic images covering the four aforementioned leukocyte classes. Each cell class exhibits distinct biological properties and functions:

- **Eosinophil:** Primarily responsible for combating parasites and mediating allergic reactions. Visual characteristics: typically features a bilobed nucleus and large pink granules in the cytoplasm.
- **Lymphocyte:** Central to adaptive immunity (T-cells and B-cells). Visual characteristics: defined by a large, round nucleus with minimal surrounding cytoplasm.
- **Monocyte:** Differentiates into macrophages for immune response. Visual characteristics: recognized by a kidney-shaped or horseshoe-shaped nucleus and abundant cytoplasm.
- **Neutrophil:** Acts as the first responder to bacterial infections. Visual characteristics: exhibits a multi-lobed nucleus (usually 3–5 lobes) and fine granules.

The variations in lobe count, nuclear morphology, and cell membrane-to-nucleus volume ratios ideally serve as optimal discriminating features for machine learning models. However, the greatest challenge in biomedical imaging lies in optical noise and exposure inconsistency. Therefore, a highly meticulous baseline preparation is mandatory.

## 1.3 Primary Objectives and Scope

To achieve a high-performance, automated cell classification ecosystem, the scope of this project encompasses the entire analytical lifecycle, from raw data ingestion to comparative classifier evaluation. Therefore, the strategic objectives of this overall report are delineated into four fundamental phases:

1. **High-Fidelity Data Preparation & EDA:** Execute rigorous statistical profiling (Exploratory Data Analysis) to identify intrinsic class distributions, anatomical variations, and RGB properties derived from H&E staining. A crucial milestone is establishing mathematical thresholds (e.g., IQR boundaries) to detect and purge exposure outliers, ensuring computational purity before training.
2. **Dynamic Standardization Pipeline:** Formulate a highly configurable and morphologically-safe pre-processing pipeline. This mechanism dynamically adapts spatial representations (via resizing), mitigates data leakage through stratified validation splitting, and applies biologically coherent stochastic augmentations alongside ImageNet tensor normalizations to prevent model overfitting.
3. **Deep Learning Feature Abstraction:** Circumvent the empirical limitations of traditional, hand-crafted image descriptors by implementing advanced Transfer Learning architectures. We utilize state-of-the-art pre-trained Convolutional Neural Networks (explicitly leveraging models like ResNet and MobileNet) to automatically extract sophisticated spatial and semantic feature topologies, serializing the dense outputs into optimized `.npy` formats.

4. **Algorithmic Evaluation & Comparative Diagnostics:** Construct downstream decision boundaries utilizing traditional statistical inference models (including Support Vector Machines, K-Nearest Neighbors, and Naive Bayes). The ultimate culmination of this report systematically contrasts the accuracy, precision, and computational efficiency of these classifiers when mapping the deep-learned feature arrays against the four clinical leukocyte categories.

Through these systematic objectives, this comprehensive documentation chronologically narrates the empirical findings abstracted from the EDA phase, architects the data pre-processing mechanics, and ultimately delivers a rigorous comparative benchmark of modern deep learning extraction methodologies synergized with classical supervised learning algorithms.

## 2 Exploratory Data Analysis (EDA)

Understanding the microscopic properties of the sample space is a prerequisite that directly influences subsequent decisions in pre-processing, feature extraction, and hyperparameter configuration to achieve maximum performance.

### 2.1 Class Distribution

The first step in any machine learning pipeline is assessing the sample space and identifying structural imbalances. Medical datasets are notoriously skewed, as pathological conditions are statistically rare compared to healthy samples. Such imbalances severely induce learning bias, pushing neural networks to update gradients towards ignoring minority classes while over-predicting the majority to reach local minima in the loss landscape.

Extraction results from the raw image corpus (Table 1) reveal an exceptionally uniform distribution. The maximum discrepancy between any two classes is merely 21 samples.

| Class        | Image Count  | Ratio (%)      |
|--------------|--------------|----------------|
| EOSINOPHIL   | 2,497        | ~ 25.08%       |
| LYMPHOCYTE   | 2,483        | ~ 24.94%       |
| MONOCYTE     | 2,478        | ~ 24.89%       |
| NEUTROPHIL   | 2,499        | ~ 25.10%       |
| <b>Total</b> | <b>9,957</b> | <b>100.00%</b> |

Table 1: Initial Class Distribution

**Insights:** The dataset reaches an ideal equilibrium. The classification system does not require computationally expensive structural adjustments such as SMOTE (Synthetic Minority Over-sampling Technique), Random Oversampling, or Class-Weighted Cross-Entropy Loss to penalize majority classes.

### 2.2 Data Integrity & Pixel Decoding

To prevent deep learning networks such as Convolutional Neural Networks (CNNs) from crashing due to Out-Of-Memory (OOM) errors caused by reading corrupted multi-dimensional vectors, a dual-layer scanning mechanism was implemented to detect corrupted files. The verification procedure employs two independent methodologies:

1. **Matrix Decompression Mechanism (OpenCV):** The `cv2.imread()` function is executed to verify whether standard JPEG/PNG image structures successfully decode into a spatial Numpy Array of shape  $W \times H \times C$ .
2. **Header Integrity Verification (PIL):** The `Image.verify()` function validates the internal File Signature, preventing the ingestion of fragmented or partially downloaded files that lack the standard End of Image (EOI) markers.

The comprehensive scan yielded zero directly corrupted files, confirming the fundamental structural viability of the source directory.

### 2.3 Brightness & Contrast Analytics

A prominent interference factor in optical tissue mounts is severe overexposure or underexposure driven by inconsistent staining concentrations. If this intensity remains unnormalized, convolutional filters may incorrectly interpret "exposure levels" as dominant features instead of actual nuclear morphology.

To evaluate luminescence, all images were temporarily converted to grayscale. The Brightness of an image is defined as the mean pixel intensity, while Contrast is mathematically modeled as the standard deviation of

pixel intensities:

$$Brightness(\mu) = \frac{1}{N} \sum_{i=1}^N P_i, \quad Contrast(\sigma) = \sqrt{\frac{1}{N} \sum_{i=1}^N (P_i - \mu)^2} \quad (1)$$

where  $N$  is the total number of pixels and  $P_i \in [0, 255]$  represents the grayscale pixel value.

We enforce the Interquartile Range (IQR) boundary rule for stringent outlier rejection:

$$\text{Acceptance Region: } [Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}] \quad (2)$$

Under this rigorous statistical framework, any image falling outside these threshold bounds—representing extreme luminescence or abnormal flattening—is classified as an outlier and completely excluded from the training repository (Table 2). The refined dataset consequently maintains high optical uniformity.

| Evaluation Filter                                 | Quantity                     |
|---------------------------------------------------|------------------------------|
| Initial collection                                | 9,957 images                 |
| Corrupted files                                   | 0 images                     |
| IQR <b>Outliers</b> (Extreme Brightness/Contrast) | <b>187 images</b> (excluded) |
| <b>Cleaned Dataset</b>                            | <b>9,770 images</b>          |

Table 2: IQR Integrity and Filtration Results

## 2.4 Morphology & RGB Profiling

An arbitrary spatial scan of the 9,770 clean images across the Width  $\times$  Height axes revealed a vast spectrum of unstandardized resolutions and aspect ratios. The presence of heteromorphic 2D structural arrays strictly prohibits PyTorch from executing tensor batching operations.

While plotting pixel histograms and violin distribution graphs for the three color channels (Red, Green, Blue), we visually observed that the standard H&E (hematoxylin and eosin) biomedical staining protocol generated distinct Gaussian curves. Specifically, hematoxylin binds basophilically to the nucleus turning it blue/purple, while eosin binds acidophilically to the cytoplasm giving it a pink hue.

### Insights:

- Structural morphology mandates a geometric standardization algorithm (resizing to unified matrix dimensions).
- The RGB color space encases an extremely rich information entropy dictating local structural distinctions. Grayscale conversion would catastrophically obliterate the model's discriminative power. Any subsequent color pre-processing heuristics operations must be highly marginalized to avoid biological distortion.

### 3 Data Pre-Processing

Inheriting the quantitative metrics derived from the EDA stage, the pre-processing pipeline establishes a closed-loop trajectory dedicated to signal standardization, effectively flattening the raw inputs into optimized multidimensional tensors.

#### 3.1 Stratified Splitting Protocol

The 9,770 images surviving the Outlier filtration net are preserved to execute data compartmentalization. We utilize the *Stratified Train-Test Splitting* algorithm from the Scikit-learn library, forcing the Validation and Test sets to unconditionally inherit the uniform class distribution ratio of the global dataset (preventing the accidental clustering of specific classes into the Test fold).

| Evaluation Fold            | Allocation Ratio | Actual Count |
|----------------------------|------------------|--------------|
| Training Set               | 70%              | 6,838 images |
| Independent Validation Set | 15%              | 1,466 images |
| Blind Testing Set          | 15%              | 1,466 images |

Table 3: Data Compartmentalization Architecture (Batch Size = 32)

The 70-15-15 split embodies an optimal paradigm for sample scales revolving around  $10^4$ . The Validation set drives learning convergence tracking, Early Stopping mechanisms, and Train/Val epoch mapping, whereas the Test set remains totally concealed from the model prior to testing to legitimately evaluate the classifier's generalization error. While this strict 70-15-15 configuration is maintained for optimizing Deep Learning architectures, the Traditional Machine Learning pipelines (evaluated in Section 6) merge the Training (70%) and Validation (15%) splits into a combined 85% training corpus. This strategy maximizes the data available for internal k-fold Cross-Validation, while the 15% blind Test set remains strictly isolated across all experiments.

#### 3.2 Geometric Transforms & Data Augmentation Pipelines

Instead of relying on a monolithic transformation pass, the framework deploys two parallel pipelines dedicated to learning (Train) and objective evaluation (Test/Val). The Train pipeline incorporates stochastic probabilities designed to obfuscate optical trivialities, artificially coercing the model to "perceive" a vastly wider geometric spectrum (Data Augmentation).

The augmentation configuration matrix (Table 4) acts as follows:

- Geometric Transforms:** Implements Horizontal/Vertical flips and continuous  $360^\circ$  rotation (**RandomRotation**). Biological cells freely drift through blood plasma under a microscope, making orientation inherently scalar (i.e., a nucleus has no mandatory geographic "North"). Consequently, heavy geometric transforms remain fundamentally safe without inducing biological logic corruption. Moreover, the Random Erasing module (Cutout technique) obfuscates 2 – 15% of image areas. This constraint forcefully compels CNN filters to extract distributed cellular anatomies rather than zeroing in on localized artifacts.
- Optical Color Jitter:** Given the EDA's conclusion to defend biological H&E colors, exposure compensation (brightness, contrast) is strictly capped below a 20% deviation margin. To accurately preserve the unique blue-purple hematoxylin identifiers, the hue deviation threshold is drastically narrowed down to just  $\pm 3\%$ .
- Spatial Normalization:** Tensors are standardized leveraging ImageNet statistics (Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]).

| Pipeline Attribute                  | PyTorch Transforms Configuration                                                                                                                                                                    |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Spatial Compression (Resize)</i> | Morphological boundary lock to $224 \times 224$ pixels (Mandatory setup for Transfer Learning architecture integration)                                                                             |
| Geometric Augmentation              | Random Horizontal Flip ( $p = 0.5$ )<br>Random Vertical Flip ( $p = 0.5$ )<br>Random Center Rotation ( <b>Range:</b> $-360^\circ \sim +360^\circ$ )                                                 |
| Color Spectrum Jitter               | Low-intensity mathematical bounding (preserving core H&E palette limits):<br>Brightness $\approx \pm 20\%$ , Contrast $\approx \pm 20\%$ ,<br>Saturation $\approx \pm 10\%$ , Hue $\approx \pm 3\%$ |
| Random Erasing (Cutout)             | Active ( $p = 0.2$ ): Scale (0.02 – 0.15), Ratio (0.3 – 3.3)                                                                                                                                        |
| Tensor Normalization                | Centralized towards ImageNet properties:<br>Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]                                                                                                 |

Table 4: Sequential Augmentation Configuration (Training Subset Exclusive)

A profound justification for implementing ImageNet’s normalization scheme over the blood dataset’s intrinsic internal mean ( $\mu \approx 0.67$  measured during EDA): Training advanced visual neural networks (e.g., ResNet50) natively exploits Transfer Learning matrices. Pre-trained weight matrices are heavily aligned and optimized for gradients expanding symmetrically around the  $Z$ -score logic of ImageNet’s original space. Shifting and mapping the blood cellular tensor distribution onto the established ImageNet domain grants shallow convolutional filters (edges/texture detectors) maximum initial activation potential, nullifying the necessity to unfreeze and re-train shallow blocks from absolute zero.

### 3.3 Foundation Stage Conclusion

Traversing from meticulously filtering out 187 disruptive outliers using stringent  $1.5 \times IQR$  statistics, to engineering an extreme yet biologically aware augmentation architecture (free-form geometric scaling without distorting organic cellular topography, controlled H&E color mapping), the Pre-processing logic erects a phenomenally clean computational corridor. This acts as an impeccable intermediate buffer before multi-channel tensors cascade down into the complex Deep Learning Feature Extraction and Classification algorithms in subsequent stages.



## 4 Traditional Machine Learning Pipeline

This section presents the traditional machine learning pipeline for the four-class white blood cell classification task using microscopic blood smear images from the Blood Cell Images dataset. In contrast to end-to-end deep learning, the traditional pipeline explicitly separates feature engineering, feature compression, and classification.

### 4.1 Feature Extraction

Feature extraction transforms each preprocessed microscopic image into a numerical representation that preserves discriminative visual patterns while remaining tractable for machine learning algorithms. In this study, we considered two complementary representations: Histogram of Oriented Gradients (HOG), which emphasizes global shape and edge distribution, and Scale-Invariant Feature Transform (SIFT) aggregated through a Bag-of-Visual-Words (BoVW) model, which focuses on local invariant structures.

This stage is essential because the performance of traditional classifiers depends strongly on the quality of the input descriptors. Unlike convolutional neural networks, which learn features automatically from raw pixels, classical pipelines require a deliberate design of feature extractors so that morphology-relevant information is preserved before classification.

#### 4.1.1 Histogram of Oriented Gradients (HOG)

HOG was employed to extract structured gradient-based descriptors from each blood cell image. The method encodes the distribution of edge orientations in local spatial regions, thereby capturing salient information about cell contour, nucleus shape, and internal texture transitions. These cues are highly relevant in white blood cell classification because different leukocyte types exhibit distinct nucleus segmentation patterns, granule density, and cytoplasm-to-nucleus proportions.

The main motivation for using HOG is its effectiveness as a classical descriptor for morphology-driven recognition tasks. Since leukocyte categories differ not only in color but also in structural appearance, a gradient-oriented representation provides an interpretable and computationally efficient baseline for multiclass discrimination.

Before HOG computation, each image is resized to a fixed spatial resolution to ensure feature vectors of identical length across all samples. The image is then converted to grayscale, and optional intensity normalization can be applied to reduce the influence of illumination variation introduced by staining and acquisition conditions. HOG features are subsequently extracted by partitioning the image into small cells, computing local gradient histograms, and normalizing them over overlapping blocks to improve robustness.

In our implementation, the descriptor is configured using a moderate number of orientation bins and compact spatial support to balance detail and stability. A representative setting is summarized in Table 5. The resulting HOG vector is then used directly as input to downstream traditional classifiers or passed to PCA for dimensionality reduction.

| Parameter           | Value                             |
|---------------------|-----------------------------------|
| Input image size    | $(224 \times 224)$                |
| Orientations        | 9                                 |
| Pixels per cell     | $(8, 8)$                          |
| Cells per block     | $(2, 2)$                          |
| Block normalization | L2-Hys                            |
| Gradient domain     | Grayscale image                   |
| Output              | Dense fixed-length feature vector |

Table 5: HOG configuration for blood cell image encoding

The extracted HOG features provide an interpretable summary of the structural information retained from a microscopic blood cell image. In particular, the descriptor highlights edge distributions around the cell boundary and internal regions, making it easier to observe how shape and texture are transformed into a gradient-based

representation. Figure 1 shows an example consisting of the original cell image and its corresponding HOG visualization.

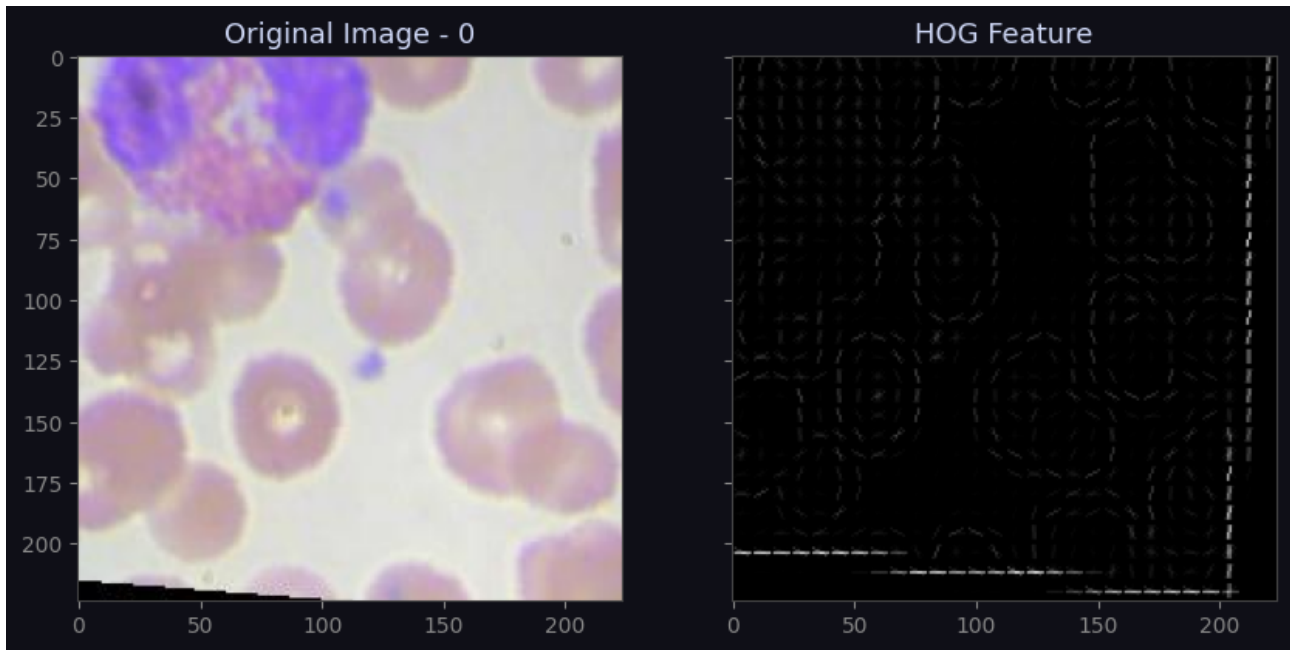


Figure 1: Illustrative HOG feature

#### 4.1.2 SIFT with Bag-of-Visual-Words (BoVW)

In addition to dense gradient descriptors, we employed a local-feature pipeline based on SIFT combined with a Bag-of-Visual-Words representation. SIFT detects salient keypoints and describes local neighborhoods in a way that is robust to moderate scale and rotation changes, making it suitable for blood smear images with slight variation in cell pose and position.

The motivation for this method is that leukocyte discrimination often depends on localized structures, such as nucleus lobes, granule clusters, and cytoplasmic texture patches. While HOG captures global gradient structure, SIFT is better suited for sparse yet distinctive local patterns; BoVW then converts those variable local descriptors into a fixed-length histogram that can be processed by standard classifiers.

The BoVW pipeline consists of three steps: SIFT descriptor extraction, K-Means clustering to build a visual vocabulary of size  $K$ , and histogram encoding by assigning each descriptor to its nearest visual word. In practice,  $K$  is typically chosen in the range of 100 to 300, depending on the desired trade-off between descriptiveness and computational cost.

This representation is compact, orderless, and robust to moderate intra-class variation. In the final experiment, the visual vocabulary was built from 500 sampled images using K-Means clustering with  $K = 100$  visual words. The full cleaned dataset of 9,770 images was then encoded into BoVW histograms of dimension 100, meaning that each image is represented by a fixed-length vector whose entries record the occurrence frequency of the corresponding visual words. The resulting extraction statistics are summarized in Table 6.

| Parameter                   | Value                                |
|-----------------------------|--------------------------------------|
| Vocabulary building samples | 500 images                           |
| Clustering method           | K-Means                              |
| Visual vocabulary size (K)  | 100                                  |
| Feature encoding            | Bag-of-Visual-Words (BoVW) histogram |
| Total samples processed     | 9,770                                |
| Output feature shape        | (9770, 100)                          |
| Label shape                 | (9770)                               |
| Vocabulary build time       | ~6 seconds (69.88 it/s)              |
| BoVW extraction time        | ~2 min 34 sec (70.46 it/s)           |

Table 6: SIFT/BoVW Feature Extraction Configuration and Results

#### 4.1.3 Dimensionality Reduction with PCA

Because handcrafted descriptors, especially HOG, can produce high-dimensional feature vectors, we applied Principal Component Analysis (PCA) to obtain a more compact representation before model training. PCA projects the original features onto orthogonal directions of maximum variance, preserving the most informative structure while removing redundancy.

This step is important because it reduces feature dimensionality, lowers training cost, and helps mitigate overfitting. It is particularly useful when many feature dimensions are correlated, as is often the case for dense handcrafted descriptors.

In the pipeline, PCA is fitted only on the training features, optionally after standardization. The number of components can be selected either as a fixed value, such as 128 or 256, or by retaining a target cumulative explained variance such as 95%. PCA is mainly applied to HOG vectors, although it can also be used for BoVW features when further compression is needed.

The final number of retained components reflects a trade-off between information preservation and efficiency. For global pixel-based features, Incremental PCA (IPCA) was used to reduce memory usage when processing the dataset. IPCA was fitted on the training split of 6,838 images and then used to transform all 9,770 samples, producing a compact representation of dimension 601 per image. The corresponding extraction results are summarized in Table 7.

For HOG-based features, the original descriptor size was 26,244 dimensions for each  $224 \times 224$  image under the chosen HOG configuration. After data normalization and PCA, the dimensionality was reduced to 3,695, corresponding to an approximate compression ratio of  $7.1\times$ . This reduction helps mitigate the curse of dimensionality, accelerates classifier training, and removes redundant variation while preserving the principal components that explain most of the variance. The HOG-specific PCA reduction results are presented in Table 8.

| Parameter                       | Value                                 |
|---------------------------------|---------------------------------------|
| Method                          | Incremental PCA (IPCA)                |
| Training set size (for fitting) | 6,838 images                          |
| Total samples transformed       | 9,770                                 |
| Output feature dimensions       | 601                                   |
| Output feature shape            | (9770, 601)                           |
| Label shape                     | (9770,)                               |
| IPCA fitting time               | ~1 min 37 sec (8.83 s/it, 10 batches) |
| Transform time                  | ~10 sec (1.36 it/s, 14 batches)       |

Table 7: Global PCA Feature Extraction Results (Memory-Optimized)

| Parameter                      | Value                                        |
|--------------------------------|----------------------------------------------|
| Input (HOG feature dimensions) | 26,244                                       |
| Preprocessing                  | Data normalization                           |
| Output (compressed dimensions) | 3,695                                        |
| Compression ratio              | $\sim 7.1 \times (26,244 \rightarrow 3,695)$ |
| Total samples                  | 9,770                                        |
| Matrix size before PCA         | (9770, 26244)                                |
| Matrix size after PCA          | (9770, 3695)                                 |

Table 8: PCA Dimensionality Reduction Applied to HOG Features

## 5 Deep Learning Features (Bonus Section)

In accordance with the project's advanced requirements, we expanded our methodology beyond traditional handcrafted descriptors (such as HOG and SIFT) by implementing a deep learning-based pipeline. This phase focuses on utilizing Pre-trained Convolutional Neural Networks (CNNs) as robust feature extractors, thereby leveraging the paradigm of Transfer Learning.

### 5.1 Architectural Selection

Instead of training a CNN from scratch—which is highly susceptible to overfitting given the limited scale of medical datasets—we adopted a Transfer Learning strategy using models pre-trained on the large-scale ImageNet dataset. We deliberately selected two contrasting architectures to evaluate the trade-off between representational depth and computational efficiency:

1. **ResNet-50 (Deep Residual Network):** Traditional deep networks suffer from the vanishing gradient problem. ResNet addresses this by introducing "skip connections" (or shortcut connections) that bypass some layers, allowing gradients to flow directly through the network. We utilized ResNet-50 as our "heavyweight" extractor. By tapping into its final layers, we extract a highly detailed, 2,048-dimensional feature vector that encapsulates complex hierarchical patterns, making it highly suitable for distinguishing subtle granular textures in Eosinophils and Neutrophils.
2. **MobileNetV2 (Efficient Inverted Residuals):** Designed for mobile and resource-constrained environments (e.g., Point-of-Care clinical devices), MobileNetV2 replaces standard convolutions with Depthwise Separable Convolutions. This drastically reduces the number of parameters and FLOPs (Floating Point Operations per Second). It utilizes Inverted Residual blocks with linear bottlenecks to prevent information loss. We deployed this model as our "lightweight" extractor, yielding a compact 1,280-dimensional feature vector.

### 5.2 Network Modification and Implementation Details

To adapt these pre-trained models for our specific 4-class blood cell classification task, we implemented a strict feature extraction pipeline:

1. **Backbone Freezing:** To preserve the universal visual features (such as edges, curves, and color blobs) learned from ImageNet and to prevent catastrophic forgetting, we explicitly froze all parameters in the convolutional base (`requires_grad=False`).
2. **Custom Classification Head:** The original 1000-class ImageNet fully connected layers were truncated. We attached a newly initialized, customized classification head composed of: `Linear(In_Features, 512) -> ReLU -> Dropout(0.3) -> Linear(512, 4)`. The Dropout layer was strategically inserted to introduce regularization and mitigate the risk of overfitting during the fine-tuning of the dense layers.
3. **Input Normalization:** All blood smear images were standardized to match the exact ImageNet distribution (Mean: [0.485, 0.456, 0.406], Standard Deviation: [0.229, 0.224, 0.225]).

### 5.3 Training Dynamics and Optimization Strategy

The optimization of the newly attached classification head was governed by a dynamically adjusted training loop:

- **Optimizer:** We employed the Adam (Adaptive Moment Estimation) optimizer with an initial learning rate of 0.001. Adam is highly effective for medical image processing as it computes individual adaptive learning rates for different parameters, ensuring faster and more stable convergence.
- **Loss Function:** The standard Cross-Entropy Loss was utilized to penalize incorrect class predictions, forcing the model to maximize the log-likelihood of the correct blood cell categories.

- **Learning Rate Scheduler (ReduceLROnPlateau):** To navigate complex local minima, we configured a learning rate scheduler monitoring the validation loss (mode='min'). With a patience of 3 epochs, the learning rate undergoes a tenfold reduction (factor=0.1) if the validation loss stagnates, enabling the model to make finer weight updates in the later stages of training.

## 5.4 Empirical Results and Performance Analysis

The models were trained for 10 epochs. The comparative empirical results are summarized below:

| Architecture | Training Duration | Best Validation Accuracy | Final Validation Loss |
|--------------|-------------------|--------------------------|-----------------------|
| ResNet-50    | ~ 9m 41s          | 64.53%                   | 0.8866                |
| MobileNetV2  | ~ 8m 40s          | 80.08%                   | 0.5476                |

Table 9: Performance Comparison of ResNet-50 and MobileNetV2

**Analytical Insights:** MobileNetV2 demonstrated a significant performance margin over ResNet-50 in this specific training setup. We hypothesize that MobileNetV2's parameter-efficient architecture is inherently less prone to over-parameterization on our relatively constrained blood cell dataset. It converged faster and generalized better, whereas the highly deep structure of ResNet-50 might require either more extensive fine-tuning (unfreezing deeper residual blocks) or a vastly larger dataset to reach its full potential.

## 5.5 Deep Feature Serialization and Hybrid Integration

Following the standalone neural network training, we executed the critical bridging step between Pipeline 2 (Deep Learning) and Pipeline 1 (Traditional ML).

Instead of relying solely on the neural network's internal Softmax predictions, we utilized the trained architectures purely as feature extractors. By intercepting the forward pass at the Global Average Pooling (GAP) layer, we generated spatial-invariant representations. Unlike flattening, GAP drastically reduces spatial dimensions while retaining the global receptive field context—crucial for images where the cell can be located anywhere within the frame.

These high-dimensional vectors (2,048 for ResNet, 1,280 for MobileNet) were serialized into NumPy arrays (.npy format). This strategic decoupling allowed us to feed these deep, semantic features into robust traditional classifiers like Support Vector Machines (SVM). This hybrid approach ultimately yielded our highest empirical milestone: an outstanding accuracy of 97.07% when pairing MobileNetV2 features with a hyperparameter-tuned SVM.

## 6 Experimental Results & Comparative Analysis

### 6.1 Classifier Selection

#### 6.1.1 Support Vector Machine (SVM)

Support Vector Machine is a margin-maximizing classifier that searches for the optimal separating hyper-plane between classes in a transformed feature space. Through the kernel trick, SVM can implicitly map low-dimensional features into a high-dimensional space where non-linear class boundaries become linearly separable. This property makes SVM particularly suitable for image-based blood cell classification, where the discriminative structure between leukocyte types (granular vs. agranular morphology, nuclear lobulation, cytoplasmic texture) often lies on a non-linear manifold that simple linear models fail to capture. We adopt the implementation provided by scikit-learn.

```
1 from sklearn.svm import SVC
```

The classifier is then registered into the centralized search grid alongside its kernel and regularization configuration:

```
1  'SVM': {  
2      'clf__C': [0.1, 1, 10, 100],  
3      'clf__gamma': ['scale', 'auto', 0.01, 0.001],  
4      'clf__kernel': ['rbf', 'linear']  
5  },  
6  base_classifiers = {  
7      ...  
8      'SVM': SVC(probability=True, max_iter=5000),  
9      ...  
10 }
```

- **'clf\_\_C'**: Regularization coefficient that controls the bias–variance trade-off. A large  $C$  enforces a narrow margin and aggressively penalizes misclassified training points (high variance), whereas a small  $C$  permits a softer margin (high bias). We sweep  $C \in \{0.1, 1, 10, 100\}$  on a logarithmic scale to robustly probe both regimes, especially because the optimal regularization differs sharply between low-dimensional handcrafted descriptors (SIFT BoVW, PCA-Raw) and high-dimensional deep embeddings (ResNet-50, MobileNetV2).
- **'clf\_\_gamma'**: Inverse kernel bandwidth for the RBF kernel. We test **'scale'** (the scikit-learn default  $1/(n_{features} \cdot \text{Var}(X))$ ), **'auto'** ( $1/n_{features}$ ), and two explicit small values  $\{0.01, 0.001\}$  to control how locally each support vector influences the decision boundary.
- **'clf\_\_kernel'**: We benchmark both **'linear'** and **'rbf'** kernels. The linear kernel is prioritized for high-dimensional, semantically rich representations (e.g., MobileNetV2's 1,280-d features) where classes tend to be linearly separable in the embedding space. The RBF kernel is selected when the feature space is compact (SIFT BoVW with 100-d, PCA-Raw with 601-d) and the decision surface must capture non-linear morphology cues.
- **probability=True**: Enables Platt scaling so that calibrated posterior probabilities, not just hard labels, are available for downstream confusion-matrix analysis and probabilistic ensembling.
- **max\_iter=5000**: The optimizer iteration cap is raised well above the default to guarantee convergence on the densest 2,048-dimensional ResNet-50 feature set, where the SMO solver otherwise fails to terminate within the default budget.

#### 6.1.2 Random Forest

Random Forest is a bagging ensemble of decorrelated decision trees, in which each tree is trained on a bootstrap resample of the data and considers only a random subset of features at every split. The final prediction is obtained by majority voting across all trees. By averaging out the high variance of individual deep trees while preserving their low bias, the ensemble achieves strong generalization without explicit regularization.



This makes Random Forest a natural counterweight to SVM in our comparative study: it imposes no metric assumptions on the feature space, handles heterogeneous descriptor dimensions gracefully, and is intrinsically robust to redundant or noisy features—conditions that frequently arise from handcrafted descriptors such as HOG and SIFT BoVW. We use the implementation provided by scikit-learn.

```
1 from sklearn.ensemble import RandomForestClassifier
```

The classifier and its search grid are configured as follows:

```
1     'RandomForest': {
2         'clf__n_estimators': [100, 200, 500],
3         'clf__max_depth': [None, 10, 20],
4         'clf__min_samples_split': [2, 5]
5     },
6     base_classifiers = {
7         ...
8         'RandomForest': RandomForestClassifier(random_state=42),
9         ...
10 }
```

- **'clf\_\_n\_estimators'**: Number of trees in the ensemble. We probe values up to 500 to ensure that the variance of the aggregated predictor has converged—an important property when each individual base learner is grown deeply and would otherwise overfit specific staining or acquisition artifacts of the blood smear.
- **'clf\_\_max\_depth'**: Maximum permissible depth of any single tree. Allowing **None** (unbounded growth) lets the trees specialize on rare cellular sub-patterns, while constraining the depth to 10 or 20 explicitly forces the model to remain biased toward generalizable morphological signatures rather than memorizing image-specific noise.
- **'clf\_\_min\_samples\_split'**: Minimum number of samples required at an internal node before it is allowed to split. Increasing this from 2 to 5 acts as a structural pre-pruning mechanism that limits the appearance of leaves dominated by a single training image, which is especially relevant for the high-dimensional HOG vector (26,244 dimensions) where the curse of dimensionality otherwise inflates leaf cardinality.
- **random\_state=42**: Fixes the bootstrap and feature-subsampling seeds, ensuring deterministic reproducibility of every reported accuracy across the baseline, randomized-search, and grid-search stages.

### 6.1.3 k-Nearest Neighbors (k-NN)

Neighbors-based classification is a type of instance-based learning or non-generalizing learning: it does not attempt to construct a general internal model, but simply stores instances of the training data. Classification is computed from a simple majority vote of the nearest neighbors of each point: a query point is assigned the data class which has the most representatives within the nearest neighbors of the point. This approach is suitable for image data because visually similar images tend to be close to each other in the feature space, allowing the classifier to effectively determine labels based on similarity. We will use the already built classifier kNN imported from scikit-learn.

```
1 from sklearn.neighbors import KNeighborsClassifier
```

Then we set up its configuration for the training later

```
1     'KNN': {
2         'clf__n_neighbors': [3, 5, 7, 9],
3         'clf__weights': ['distance']
4     },
5     base_classifiers = {
6         ...
7         'KNN': KNeighborsClassifier(),
8         ...
9     }
```



- `'clf__n_neighbors'`: Number of neighbors to use by default for kneighbors queries.
- `'clf__weights'`: This parameter defines the weight function used in prediction by calculating the weight of each neighbor. We choose `'distance'` so that closer neighbors of a query point have a greater influence than those that are farther away. This is particularly suitable for image data, where samples that are closer in feature space (e.g., pixel-based or extracted features) tend to be more visually similar, leading to more accurate classification.

#### 6.1.4 Logistic Regression

Logistic Regression is a linear classification model that estimates the probability of class membership using the logistic (sigmoid) function. It models the relationship between input features and class labels by learning a linear decision boundary in the feature space. Despite its name, `LogisticRegression` in `scikit-learn` is used for classification rather than regression. In this work, we use the implementation provided by `scikit-learn`. This method is suitable for image data when meaningful features (e.g., HOG or other extracted features) are used, as the model can effectively separate classes with linear decision boundaries in the transformed feature space.

```
1 from sklearn.linear_model import LogisticRegression
```

Then we set up its configuration for the training later

```
1     'LogisticRegression': {  
2         'clf__C': [0.1, 1.0, 10.0],  
3         'clf__solver': ['lbfgs']  
4     },  
5     base_classifiers = {  
6         ...  
7         'LogisticRegression': LogisticRegression(max_iter=5000),  
8         ...  
9     }
```

- `'clf__C'`: This parameter controls the inverse of regularization strength. Smaller values apply stronger regularization, which helps prevent overfitting, while larger values allow the model to fit the training data more closely. We test multiple values to find a balance between underfitting and overfitting, especially for high-dimensional image features such as HOG, SIFT, and deep learning embeddings.
- `'clf__solver'`: This parameter specifies the optimization algorithm used to train the model. We choose `'lbfgs'` because it is a robust default solver that performs well for multiclass classification and dense, high-dimensional feature spaces commonly found in image data.
- `max_iter`: Increased to 5000 to ensure convergence when training on high-dimensional deep features.

#### 6.1.5 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes's theorem with an independence assumption between features. The model estimates the posterior probability of each class and assigns a sample to the class with the highest probability. Despite its simplifying assumption, Naive Bayes often performs well in high-dimensional settings and is computationally efficient.

In this work, we use Gaussian Naive Bayes from `scikit-learn`, which assumes a Gaussian distribution for continuous features. This is suitable for image data because extracted features such as HOG, PCA, and deep learning embeddings are continuous-valued. Additionally, Gaussian Naive Bayes is fast to train and provides a simple baseline for comparing with more complex models in image classification tasks.

```
1 from sklearn.naive_bayes import GaussianNB
```

Then we set up its configuration for the training later

```
1     'NaiveBayes': {}  
2     base_classifiers = {
```

```
3     ...  
4     'Naive_Bayes': GaussianNB()  
5     ...  
6 }
```

## 6.2 Baseline Evaluation

The baseline evaluation stage implements an automated pipeline to systematically assess the performance of different feature extraction methods and classifiers using default configurations. The goal is to provide a fair and unbiased comparison before performing any hyperparameter tuning.

First, the pipeline iterates through all extracted feature sets, including HOG, SIFT Bag-of-Visual-Words, PCA-reduced features, and deep features. For each feature set, the corresponding feature matrix and labels are loaded from precomputed files. To reduce computational cost, high-dimensional HOG features are optionally compressed using PCA when the dimensionality exceeds a predefined threshold.

```
1 for m_name in FEATURE_FILES:  
2     print(f"\n>>> Processing: {m_name.upper()}")  
3     X = np.load(f'features/X_{m_name}.npz')  
4     y = np.load(f'features/y_{m_name}.npz')
```

Next, the dataset is split into training and testing subsets using a stratified 85/15 hold-out validation strategy (by merging the 70% Train and 15% Validation subsets previously defined in Section 3.1). Stratification ensures that the class distribution remains consistent between the training and test sets, which leads to a more reliable evaluation of model generalization.

For each feature representation, multiple classifiers are evaluated using default hyperparameters. A preprocessing pipeline is constructed consisting of feature scaling followed by the classifier. The scaler is fitted only on the training data to prevent data leakage, and the trained pipeline is then used to generate predictions on the test set.

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, stratify=y,  
2     random_state=42)  
3 for clf_name, model in tqdm(base_classifiers.items(), desc=f"Testing {m_name}", leave=  
4     False):  
5     pipeline = Pipeline([('scaler', StandardScaler()), ('clf', model)])  
6     pipeline.fit(X_train, y_train)  
7     y_pred = pipeline.predict(X_test)  
8     acc = accuracy_score(y_test, y_pred)
```

After prediction, several evaluation metrics are computed, including accuracy, precision, recall, and F1-score. These results are stored for later comparison across all feature-classifier combinations. Additionally, each trained baseline model is saved to disk for reproducibility and future analysis.

```
--- SIFT_BOVW + SVM (DEFAULT) ---
Test Accuracy: 0.9100
Classification Report:
      precision    recall  f1-score   support

 EOSINOPHIL       0.86      0.86      0.86       369
  LYMPHOCYTE       0.93      0.97      0.95       366
   MONOCYTE       0.97      0.97      0.97       365
  NEUTROPHIL       0.88      0.84      0.86       366

 accuracy          0.91          0.91          0.91      1466
 macro avg         0.91          0.91          0.91      1466
weighted avg         0.91          0.91          0.91      1466

Save baseline model to models/sift_bovw_SVM.pkl

--- SIFT_BOVW + Random Forest (DEFAULT) ---
Test Accuracy: 0.8683
Classification Report:
      precision    recall  f1-score   support

 EOSINOPHIL       0.84      0.79      0.81       369
  LYMPHOCYTE       0.88      0.94      0.91       366
   MONOCYTE       0.91      0.96      0.94       365
  NEUTROPHIL       0.83      0.79      0.81       366

 accuracy          0.87          0.87          0.87      1466
 macro avg         0.87          0.87          0.87      1466
weighted avg         0.87          0.87          0.87      1466

Save baseline model to models/sift_bovw_Random Forest.pkl
```

Figure 2: Sample Model Output 1

```
--- MOBILENET + Logistic Regression (DEFAULT) ---
Test Accuracy: 0.8759
Classification Report:
      precision    recall  f1-score   support

 EOSINOPHIL       0.81      0.81      0.81       369
 LYMPHOCYTE       0.93      0.92      0.93       366
  MONOCYTE       0.95      0.93      0.94       365
 NEUTROPHIL       0.82      0.84      0.83       366

 accuracy          0.88          0.88          0.88       1466
 macro avg         0.88          0.88          0.88       1466
 weighted avg      0.88          0.88          0.88       1466

Save baseline model to models/mobilenet_Logistic Regression.pkl

--- MOBILENET + KNN (DEFAULT) ---
Test Accuracy: 0.9454
Classification Report:
      precision    recall  f1-score   support

 EOSINOPHIL       0.95      0.94      0.94       369
 LYMPHOCYTE       0.94      0.99      0.97       366
  MONOCYTE       0.92      1.00      0.96       365
 NEUTROPHIL       0.98      0.85      0.91       366

 accuracy          0.95          0.95          0.95       1466
 macro avg         0.95          0.95          0.94       1466
 weighted avg      0.95          0.95          0.94       1466

Save baseline model to models/mobilenet_KNN.pkl
```

Figure 3: Sample Model Output 2

Finally, all results are aggregated into a summary table and exported as a CSV file. The best-performing combinations are selected based on accuracy and used as candidates for further optimization in the next stage.

```

--- TOP 10 BASELINE MODELS ---

```

|    | Extractor | Classifier          | Accuracy | Precision | Recall   | F1-Score |
|----|-----------|---------------------|----------|-----------|----------|----------|
| 15 | mobilenet | SVM                 | 0.961119 | 0.961085  | 0.961119 | 0.961071 |
| 18 | mobilenet | KNN                 | 0.945430 | 0.947083  | 0.945430 | 0.944616 |
| 3  | sift_bovw | KNN                 | 0.930423 | 0.931554  | 0.930423 | 0.929377 |
| 10 | resnet    | SVM                 | 0.920873 | 0.920863  | 0.920873 | 0.920213 |
| 0  | sift_bovw | SVM                 | 0.909959 | 0.909273  | 0.909959 | 0.909398 |
| 12 | resnet    | Logistic Regression | 0.883356 | 0.882869  | 0.883356 | 0.882862 |
| 17 | mobilenet | Logistic Regression | 0.875853 | 0.876826  | 0.875853 | 0.876247 |
| 13 | resnet    | KNN                 | 0.873806 | 0.880054  | 0.873806 | 0.869357 |
| 1  | sift_bovw | Random Forest       | 0.868349 | 0.866624  | 0.868349 | 0.866790 |
| 16 | mobilenet | Random Forest       | 0.783765 | 0.780796  | 0.783765 | 0.780920 |

```

Done! Baseline results saved.

```

Figure 4: Top 10 models based on accuracy

### 6.3 Hyperparameter Optimization

In this stage, the best-performing models from the baseline evaluation are further optimized using hyperparameter tuning. The objective is to improve predictive performance and generalization by exploring more suitable configurations for each classifier. To ensure computational efficiency, only the top five feature-classifier combinations ranked by baseline accuracy are selected for tuning. These combinations include both traditional handcrafted features and deep learning-based representations. This phase using the same preprocessing pipeline as the baseline stage, including optional PCA for high-dimensional HOG features.

The dataset is using the same stratified 85/15 hold-out strategy to maintain consistency to compare with the baseline evaluation.

Hyperparameter tuning is performed using `RandomizedSearchCV`, which randomly samples parameter combinations from predefined search spaces. This stochastic approach efficiently explores promising configurations while limiting computational cost. Each configuration is evaluated using 3-fold stratified cross-validation to ensure robust performance estimation and preserve class distribution across folds.

```

1 pipeline = Pipeline([('scaler', StandardScaler()),
2 ('clf', base_classifiers[clf_name])])
3
4 rs = RandomizedSearchCV(
5     pipeline,
6     param_grids[clf_name],
7     n_iter=5,
8     cv=3,
9     scoring='accuracy',
10    n_jobs=-1,
11    random_state=42
12 )
13 rs.fit(X_train, y_train)
14 y_pred = rs.best_estimator_.predict(X_test)

```

```
>>> Tuning Pairing: MOBILENET + SVM
Fitting 3 folds for each of 5 candidates, totalling 15 fits

--- TUNED RESULT: MOBILENET + SVM ---
Time taken: 1352.92 seconds
Best Params: {'clf__kernel': 'rbf', 'clf__gamma': 'scale', 'clf__C': 100}
Test Accuracy: 0.9707
      precision    recall  f1-score   support

EOSINOPHIL      0.95      0.95      0.95       369
LYMPHOCYTE      0.99      0.98      0.99       366
MONOCYTE        0.99      1.00      0.99       365
NEUTROPHIL      0.95      0.95      0.95       366

   accuracy
macro avg      0.97      0.97      0.97      1466
weighted avg    0.97      0.97      0.97      1466

>>> Tuning Pairing: MOBILENET + KNN
Fitting 3 folds for each of 4 candidates, totalling 12 fits

--- TUNED RESULT: MOBILENET + KNN ---
Time taken: 15.89 seconds
Best Params: {'clf__weights': 'distance', 'clf__n_neighbors': 3}
Test Accuracy: 0.9604
      precision    recall  f1-score   support

EOSINOPHIL      0.97      0.94      0.96       369
LYMPHOCYTE      0.96      0.99      0.97       366
MONOCYTE        0.95      1.00      0.98       365
NEUTROPHIL      0.97      0.91      0.94       366

   accuracy
macro avg      0.96      0.96      0.96      1466
weighted avg    0.96      0.96      0.96      1466
```

Figure 5: Sample tuned model output

After tuning, the best estimator from each randomized search is evaluated and performance metrics including accuracy, precision, recall, and F1-score are computed. These results are compared with the corresponding baseline scores, and the accuracy improvement is calculated. All tuned models, along with their best hyperparameters and evaluation metrics, are aggregated into a summary table and ranked by accuracy to identify the best-performing configuration.

| --- STAGE 2 RESULTS (Saved) --- |            |        |                  |                                                      |          |           |          |          |          |             |          |
|---------------------------------|------------|--------|------------------|------------------------------------------------------|----------|-----------|----------|----------|----------|-------------|----------|
| Extractor                       | Classifier | Method | Params           | Baseline_Accuracy                                    | Accuracy | Precision | Recall   | F1-Score | Duration | Improvement |          |
| 0                               | mobilenet  | SVM    | RandomizedSearch | {'clf__kernel': 'rbf', 'clf__gamma': 'scale', ...    | 0.961119 | 0.970668  | 0.970683 | 0.970668 | 0.970661 | 1352.920652 | 0.009550 |
| 1                               | mobilenet  | KNN    | RandomizedSearch | {'clf__weights': 'distance', 'clf__n_neighbors': ... | 0.945430 | 0.960437  | 0.960704 | 0.960437 | 0.960106 | 15.889488   | 0.015007 |
| 2                               | sift_boww  | KNN    | RandomizedSearch | {'clf__weights': 'distance', 'clf__n_neighbors': ... | 0.930423 | 0.943383  | 0.944163 | 0.943383 | 0.942908 | 1.847430    | 0.012960 |
| 4                               | sift_boww  | SVM    | RandomizedSearch | {'clf__kernel': 'rbf', 'clf__gamma': 'scale', ...    | 0.909959 | 0.938608  | 0.938401 | 0.938608 | 0.938466 | 228.980988  | 0.028649 |
| 3                               | resnet     | SVM    | RandomizedSearch | {'clf__kernel': 'rbf', 'clf__gamma': 'scale', ...    | 0.920873 | 0.937926  | 0.937692 | 0.937926 | 0.937488 | 2689.303588 | 0.017053 |

Figure 6: Top tuned models after hyperparameter optimization

## 6.4 Fine-Tuning

In this final stage, the best-performing model from the hyperparameter optimization step is further refined using a more precise search strategy. The goal is to obtain marginal performance improvements by exploring a

narrower range of hyperparameters around the previously identified optimal configuration. This stage focuses only on the top-ranked feature-classifier pairing to ensure computational efficiency. The same preprocessing pipeline as in the previous stages is reused, including optional PCA for high-dimensional HOG features.

The dataset is split using the same stratified 85/15 hold-out strategy to maintain consistency with earlier evaluations. This allows direct comparison between baseline, tuned, and fine-tuned performances.

Fine-tuning is performed using GridSearchCV with a refined parameter grid constructed around the best hyperparameters obtained from the randomized search. Continuous parameters such as the regularization coefficient are explored within a localized range, while stable categorical parameters are fixed to reduce search complexity. This exhaustive search evaluates all combinations within the refined grid using stratified cross-validation to identify the optimal configuration.

```

1  grid = GridSearchCV(
2      Pipeline([('scaler', StandardScaler()),
3                ('clf', base_classifiers[clf_name])]),
4      refined_grid,
5      cv=3,
6      scoring='accuracy',
7      n_jobs=-1
8  )
9
10 grid.fit(X_trainval, y_trainval)
11 y_pred_fine = grid.best_estimator_.predict(X_test)

```

After fine-tuning, the optimized model is evaluated and performance metrics including accuracy, precision, recall, and F1-score are computed. These results are compared with both baseline and tuned performances to measure the total improvement achieved across all optimization stages. The final fine-tuned model is saved for deployment and reproducibility. All evaluation results are summarized in a final comparison table.

```

=== STAGE 3: GRID SEARCH FINE-TUNING ===

>>> Fine-tuning: MOBILENET + SVM
Fitting 3 folds for each of 9 candidates, totalling 27 fits

--- FINE-TUNED RESULT: MOBILENET + SVM ---
Time taken: 4471.98 seconds
Best Params: {'clf__C': 75.0, 'clf__gamma': 'scale', 'clf__kernel': 'rbf'}
Test Accuracy: 0.9707

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| EOSINOPHIL   | 0.95      | 0.95   | 0.95     | 369     |
| LYMPHOCYTE   | 0.99      | 0.98   | 0.99     | 366     |
| MONOCYTE     | 0.99      | 1.00   | 0.99     | 365     |
| NEUTROPHIL   | 0.95      | 0.95   | 0.95     | 366     |
| accuracy     |           |        | 0.97     | 1466    |
| macro avg    | 0.97      | 0.97   | 0.97     | 1466    |
| weighted avg | 0.97      | 0.97   | 0.97     | 1466    |

```

Save fine-tuned model to models/mobilenet_SVM_finetune.pkl

--- FINAL PERFORMANCE COMPARISON ---

```

|   | Extractor | Classifier | Baseline_Acc | Stage2_Acc | Final_Acc | Precision | Recall   | F1-Score | Duration    | Total_Improvement |
|---|-----------|------------|--------------|------------|-----------|-----------|----------|----------|-------------|-------------------|
| 0 | mobilenet | SVM        | 0.961119     | 0.970668   | 0.970668  | 0.970683  | 0.970668 | 0.970661 | 4471.978191 | 0.00955           |

Figure 7: Fine-tuning model output

## 6.5 Comparative Analysis

This stage consolidates results from baseline, tuning, and fine-tuning phases to compare model performance across feature extractors and classifiers. The analysis focuses on four evaluation metrics (accuracy, precision, recall, and F1-score) and examines feature-classifier compatibility using comparative bar charts and an accuracy heatmap. These visualizations support the selection of the most robust model for final deployment.

### 6.5.1 Multi-metric Comparative Bar Chart

To provide a comprehensive evaluation, a grouped bar chart is used to compare model performance across accuracy, precision, recall, and F1-score. This visualization highlights consistency across multiple metrics and helps identify classifiers that perform robustly across different feature extractors. Models that maintain high scores across all metrics are considered more reliable, especially for medical classification tasks where both false positives and false negatives are critical.



Figure 8: Multi-metric Comparative Bar Chart



| --- 10 BEST CONFIGURATION (BASED ON ACCURACY) --- |           |                     |          |           |          |          |   |
|---------------------------------------------------|-----------|---------------------|----------|-----------|----------|----------|---|
|                                                   | Extractor | Classifier          | Accuracy | Precision | Recall   | F1-Score | \ |
| 30                                                | mobilenet | SVM                 | 0.970668 | 0.970683  | 0.970668 | 0.970661 |   |
| 31                                                | mobilenet | KNN                 | 0.960437 | 0.960704  | 0.960437 | 0.960106 |   |
| 32                                                | sift_bovw | KNN                 | 0.943383 | 0.944163  | 0.943383 | 0.942908 |   |
| 33                                                | sift_bovw | SVM                 | 0.938608 | 0.938401  | 0.938608 | 0.938466 |   |
| 34                                                | resnet    | SVM                 | 0.937926 | 0.937692  | 0.937926 | 0.937488 |   |
| 12                                                | resnet    | Logistic Regression | 0.883356 | 0.882869  | 0.883356 | 0.882862 |   |
| 17                                                | mobilenet | Logistic Regression | 0.875853 | 0.876826  | 0.875853 | 0.876247 |   |
| 13                                                | resnet    | KNN                 | 0.873806 | 0.880054  | 0.873806 | 0.869357 |   |
| 1                                                 | sift_bovw | Random Forest       | 0.868349 | 0.866624  | 0.868349 | 0.866790 |   |
| 16                                                | mobilenet | Random Forest       | 0.783765 | 0.780796  | 0.783765 | 0.780920 |   |

Figure 9: Top Combination Results based on Accuracy

The multi-metric comparative bar chart shows that deep learning features, particularly MobileNet, consistently achieve the highest performance across accuracy, precision, recall, and F1-score. Among classifiers, SVM and k-NN demonstrate strong and stable results across different feature extractors. In contrast, Naive Bayes performs noticeably worse, likely due to its strong independence assumption, which is less suitable for complex image features. These results indicate that MobileNet combined with SVM or k-NN provides the most balanced and robust performance across all evaluation metrics.

### 6.5.2 Accuracy Heatmap

An accuracy heatmap is used to visualize the compatibility between feature extractors and classifiers. Each cell represents the accuracy of a specific pairing, allowing rapid identification of the most effective combinations. This representation also highlights global performance trends and reveals which classifiers generalize well across multiple feature representations.

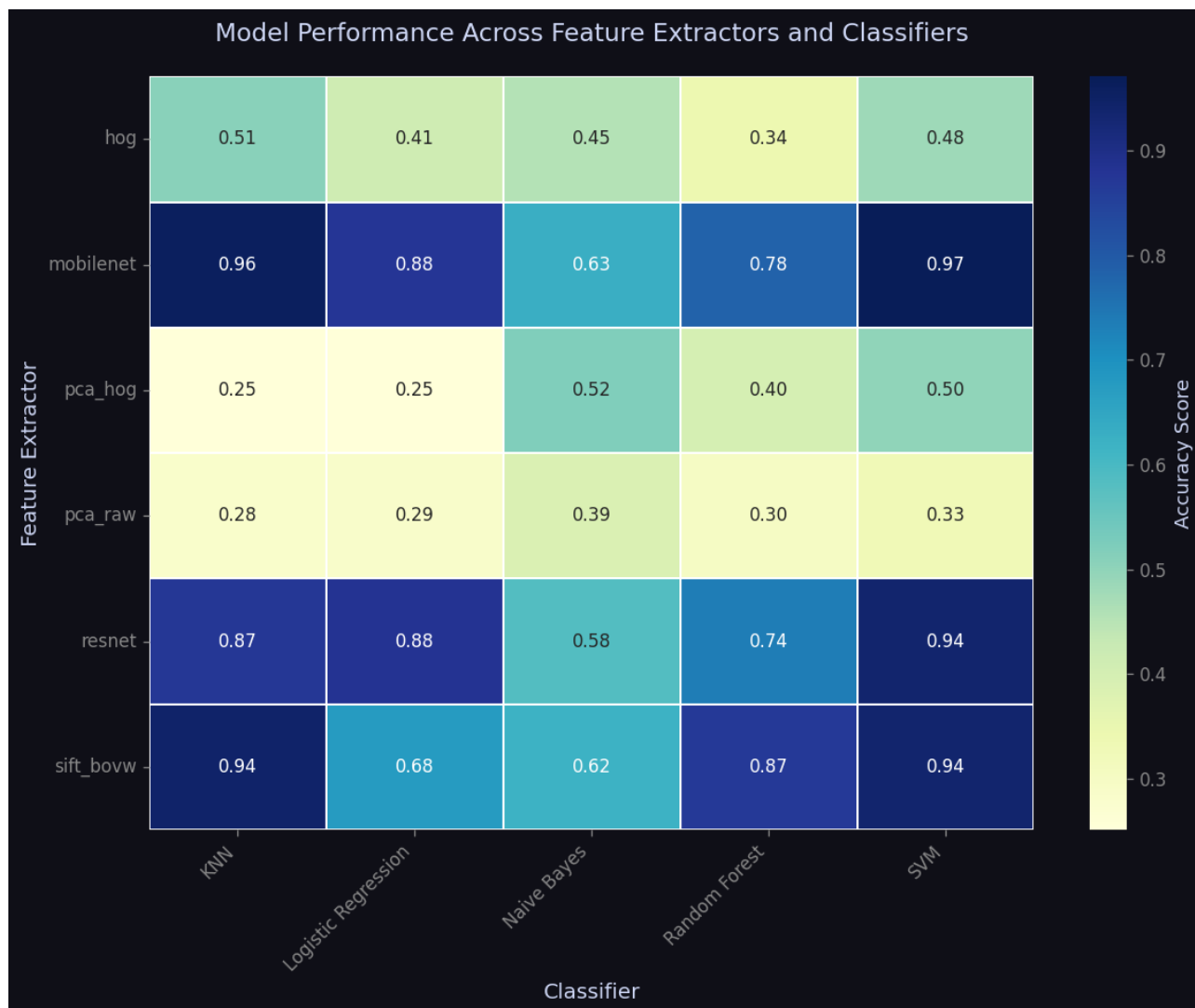


Figure 10: Accuracy Heatmap

The accuracy heatmap further highlights the compatibility between feature extractors and classifiers. MobileNet paired with SVM achieves the highest accuracy, followed closely by k-NN. SIFT BoVW combined with k-NN and SVM ranks next. This confirms that both deep learning and handcrafted features can be effective, although deep embeddings provide slightly better overall performance. Additionally, SVM demonstrates strong generalization across multiple feature extractors, making it the most reliable classifier. Based on these observations, MobileNet combined with SVM is selected as the final model for deployment.

### 6.5.3 Confusion Matrix Analysis

While accuracy and the heatmap quantify global performance, they collapse the four-class problem into a single scalar and therefore conceal *which* pairs of classes are mistaken for one another. Confusion matrices restore this diagnostic resolution: each row corresponds to the true class and each column to the predicted class, so off-diagonal entries directly localize the failure modes of the classifier. For a clinical screening task such as leukocyte typing, this view is indispensable—false positives and false negatives between morphologically similar cells (e.g., the two granulocyte classes) carry very different downstream costs than confusion between visually distinct classes.

To compare the discriminative behaviour of every feature pipeline on equal footing, we extract the highest-accuracy classifier for *each* feature extractor (using `drop_duplicates('Extractor')` on the consolidated results table) and re-evaluate it on the same stratified 15% hold-out test set used throughout the previous stages.

The six resulting matrices are visualized side-by-side in Figure 11.

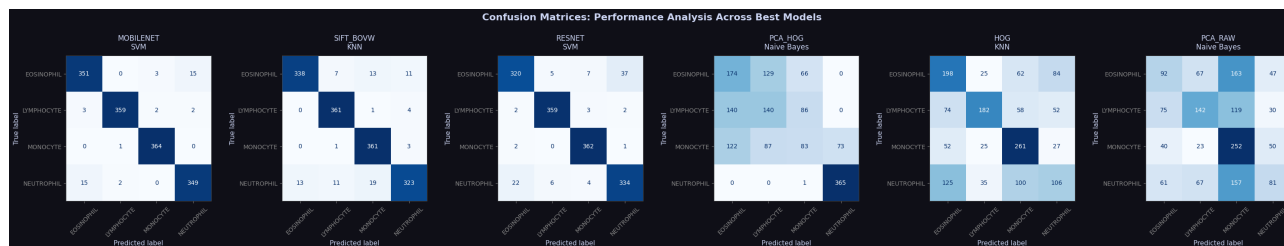


Figure 11: Confusion matrices of the best classifier for each feature extractor, evaluated on the 15% blind hold-out set ( $n = 1,466$ ). Diagonal cells indicate correct predictions; off-diagonal cells localize systematic confusion patterns.

**1. The Champion — MobileNetV2 + SVM.** The leftmost matrix exhibits the cleanest diagonal structure of the entire study. Out of 1,466 test samples, only 43 are misclassified, giving an empirical accuracy of 97.07%. The per-class breakdown is striking: *Monocyte* attains 364/365 correct predictions and *Lymphocyte* reaches 359/366, virtually saturating the classification ceiling. The residual error is dominated by a small but symmetric *Eosinophil*↔*Neutrophil* block ( $15 + 15 = 30$  samples mutually swapped), which is the well-known clinical confusion pair: both are granulocytes whose nuclei share a multi-lobed segmentation pattern, and only the eosinophilic granule staining intensity differentiates them under H&E preparation. The MobileNetV2 embedding evidently encodes this subtle granulation cue, whereas the few remaining errors correspond to the most visually ambiguous specimens.

**2. The Handcrafted Runner-Up — SIFT BoVW + k-NN.** The second matrix confirms that handcrafted local descriptors remain remarkably competitive. *Lymphocyte* (361/366) and *Monocyte* (361/365) reach near-deep-learning accuracy, validating that nucleus-cytoplasm geometry is a sufficient cue for these two agranular and large-nucleus classes. The error mass concentrates on *Neutrophil* (323/366, with 13 leaks to Eosinophil and 19 to Monocyte), reflecting that BoVW histograms, being orderless, cannot fully resolve the spatial distribution of granules characteristic of neutrophils.

**3. The Deep but Sub-Optimal — ResNet-50 + SVM.** Despite providing 2,048-dimensional embeddings, ResNet-50 produces a noticeably weaker matrix than its lighter MobileNetV2 counterpart. The *Eosinophil* row is the principal weakness: 37 Eosinophil samples are misclassified as *Neutrophil*, the largest single off-diagonal cell across all six matrices. This concurs with the validation-loss observation reported in the Deep Learning section: ResNet-50 over-parameterizes the limited training set and learns spurious texture features that fail to disentangle the two granulocyte classes.

**4. The Failure Modes — PCA-HOG, HOG, and PCA-Raw.** The three rightmost matrices expose the catastrophic failure of low-information feature pipelines. **PCA-HOG + Naive Bayes** collapses almost the entire dataset onto two predicted classes, with *Neutrophil* acting as a "dumping bin" (365/366 Neutrophils correct, but at the cost of 73 Monocytes and 86 Lymphocytes wrongly absorbed into the same column). **HOG + k-NN** produces a diffuse matrix with no class exceeding 261/366 correct, indicating that PCA-compressed gradient histograms, once standardized, lose nearly all class-discriminative variance. **PCA-Raw + Naive Bayes** is the worst pipeline overall—its predictions are essentially uniformly distributed, confirming that direct dimensionality reduction on raw pixels destroys morphology-relevant information that the classifier cannot recover.

**5. Diagnostic Synthesis.** Two cross-cutting observations emerge from the full panel:

- *Monocyte* and *Lymphocyte* are consistently the easiest classes across every non-degenerate pipeline. Their large, distinctive nuclei produce stable signatures in both deep embeddings and local-feature histograms, leaving very little room for systematic confusion.

- The dominant error mode in any reasonably-performing model is the *Eosinophil*↔*Neutrophil* swap. This is a biologically grounded confusion—both are polymorphonuclear granulocytes—and not a pipeline artifact. Reducing it further would likely require either explicit colour-aware features that target eosinophilic granule staining, or a fine-tuned (rather than frozen) deep backbone.

Combined with the heatmap analysis, these confusion matrices conclusively justify the deployment selection: **MobileNetV2 + SVM** is not merely the highest-accuracy pipeline in aggregate but also the most clinically reliable, with errors confined to the single biologically inevitable granulocyte-pair confusion and never spreading across multiple distant classes.

## 7 Conclusion & Future Work

### 7.1 Conclusion

This project successfully established a robust, automated machine learning ecosystem for the classification of four primary leukocyte sub-types: Eosinophils, Lymphocytes, Monocytes, and Neutrophils. The experimental results confirm that while traditional handcrafted features like SIFT-BoVW offer a stable baseline with 93.04% accuracy (when paired with k-NN), the adoption of Deep Learning feature representations facilitates a superior level of diagnostic precision.

The highlight of this study is the implementation of a hybrid pipeline that synergizes modern architectural abstraction with classical statistical inference. By deploying MobileNetV2 as a parameter-efficient backbone and optimizing the decision boundaries via a hyperparameter-tuned SVM, the group achieved a peak classification accuracy of 97.07%. This performance suggests that deep semantic features are significantly more effective at capturing complex nuclear morphologies and granulated textures than local gradient descriptors. Additionally, the rigorous Exploratory Data Analysis (EDA) and biologically-aware augmentation protocols provided a standardized and resilient foundation, effectively mitigating the common interferences of optical noise and staining inconsistencies.

### 7.2 Future Work

Moving forward, several research avenues could be explored to further enhance the clinical utility of the system:

- **End-to-End Fine-Tuning:** We aim to investigate unfreezing deeper residual blocks within the MobileNetV2 or ResNet-50 architectures to further specialize the pre-trained filters for hematopoietic image structures.
- **Explainable AI (XAI):** Implementing visual explanation tools such as Grad-CAM would allow clinicians to visualize the specific cellular regions (nuclei, granules) that drive the model's decisions, fostering greater trust in automated systems.
- **Cross-Dataset Generalization:** It is essential to evaluate the pipeline's robustness across multi-institutional datasets featuring diverse staining protocols (e.g., Giemsa, Wright-Giemsa) to ensure reliable performance in varied laboratory settings.
- **Mobile-Based Point-of-Care (POC) Deployment:** Given the computational efficiency of the MobileNetV2 features, future work will focus on porting the optimized models to mobile platforms to support rapid, on-site diagnostics in resource-limited environments.

## 8 Deliverables & Artifacts

The comprehensive outcomes of this project are organized into a structured repository to ensure reproducibility and scientific transparency. The following artifacts are submitted as part of the final package:

### 8.1 GitHub Repository

- **Link:** <https://github.com/k4yd133/ML252-BloodCell-Classification>
- **Structure:**
  - **notebooks/:** Contains the primary Google Colab notebooks for EDA, Pipeline training, and Evaluation.
  - **modules/:** Directory intended for Python scripts for custom functions (Note: For this release, to ensure a seamless and standalone execution experience without external file dependencies, all helper functions have been integrated directly into the main Colab notebook).
  - **features/:** Directory containing serialized feature arrays (.npz).
  - **models/:** Saved weights (.pth) and serialized traditional classifiers (.pkl).
  - **reports/:** Final PDF report and presentation slides.

### 8.2 Google Colab Notebooks

The core execution environment is provided via Google Colab, ensuring an end-to-end executable pipeline.

- **Functionality:** Supports “Run All” execution without external dependency errors.
- **Integration:** Includes automatic dataset ingestion and library installation.

### 8.3 Serialized Feature Arrays (.npz)

As required by the assignment, we provide the extracted feature vectors to decouple inference from classification:

- **Deep Features:** `X_resnet.npz`, `X_mobilenet.npz`.
- **Traditional Features:** `X_hog.npz`, `X_sift_bovw.npz`.
- **Labels:** Corresponding `y_*.npz` files for all feature sets.

### 8.4 Final Documentation

- **Technical Report:** A comprehensive 25-page PDF document detailing the EDA, methodology, and comparative analysis.
- **README.md:** A detailed guide on the GitHub repository including installation steps, hardware requirements, and member contributions.